

Informe de Laboratorio 05

Tema: RPC vs gRPC

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	<i>RPC vs gRPC</i>				
NÚMERO DE PRÁCTICA:	<i>05</i>	AÑO LECTIVO:	2026	NRO. SEMESTRE:	2026A
FECHA DE PRESENTACIÓN	<i>18/05/2026</i>	HORA DE PRESENTACIÓN	14:00		
INTEGRANTE(s): GRUPO B • Chambilla Perca Ricardo Mauricio • Gutierrez Ccama Juan Diego • Valdivia Segovia Ryan Fabian				NOTA:	
DOCENTE(s): <i>Mg. Maribel Molina Barriga</i>					

1. Repositorio de GitHub

El código fuente del laboratorio se encuentra en el repositorio: https://github.com/rikich3/LAB_SISTEMAS_DISTRIBUIDOS

2. Estructura de la sesión

- resueltos/: ejercicio resuelto por docente (Calculadora gRPC).
- propuestos/: ejercicios implementados (RMI y gRPC Converter).
- informe/: memoria técnica de la sesión.

3. Ejercicio Resuelto por el Docente

3.1. Implementación Calculadora gRPC

Listing 1: calculator.proto (resuelto)

```
1 syntax = "proto3";
2
3 option java_multiple_files = true;
4 option java_package = "edu.unsa.sd.resuelto";
5 option java_outer_classname = "CalculatorProto";
```

```
6
7 service Calculator {
8     rpc Sum (Request) returns (Response);
9 }
10
11 message Request {
12     int32 a = 1;
13     int32 b = 2;
14 }
15
16 message Response {
17     int32 result = 1;
18 }
```

Listing 2: CalculatorService.java (resuelto)

```
1 package edu.unsa.sd.resuelto;
2
3 import io.grpc.stub.StreamObserver;
4
5 public class CalculatorService extends CalculatorGrpc.CalculatorImplBase {
6     @Override
7     public void sum(Request req, StreamObserver<Response> responseObserver) {
8         int result = req.getA() + req.getB();
9         Response response = Response.newBuilder().setResult(result).build();
10        responseObserver.onNext(response);
11        responseObserver.onCompleted();
12    }
13 }
```

Listing 3: CalculatorServer.java (resuelto)

```
1 package edu.unsa.sd.resuelto;
2
3 import io.grpc.Server;
4 import io.grpc.ServerBuilder;
5
6 public class CalculatorServer {
7     public static void main(String[] args) throws Exception {
8         Server server = ServerBuilder.forPort(50051)
9             .addService(new CalculatorService())
10            .build()
11            .start();
12
13        System.out.println("Servidor gRPC Calculator iniciado en puerto 50051");
14        server.awaitTermination();
15    }
16 }
```

Listing 4: CalculatorClient.java (resuelto)

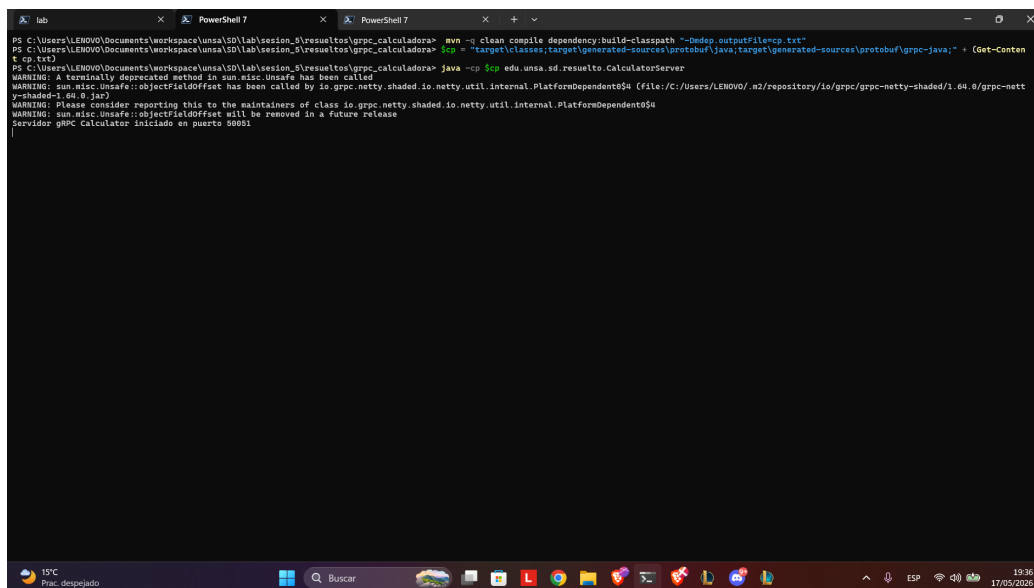
```
1 package edu.unsa.sd.resuelto;
2
3 import io.grpc.ManagedChannel;
4 import io.grpc.ManagedChannelBuilder;
```

```

5
6 public class CalculatorClient {
7     public static void main(String[] args) {
8         ManagedChannel channel = ManagedChannelBuilder.forAddress("localhost", 50051)
9             .usePlaintext()
10            .build();
11
12        CalculatorGrpc.CalculatorBlockingStub stub = CalculatorGrpc.newBlockingStub(channel);
13
14        Request request = Request.newBuilder().setA(8).setB(4).build();
15        Response response = stub.sum(request);
16
17        System.out.println("Resultado: " + response.getResult());
18        channel.shutdown();
19    }
20 }

```

3.2. Evidencias de ejecución

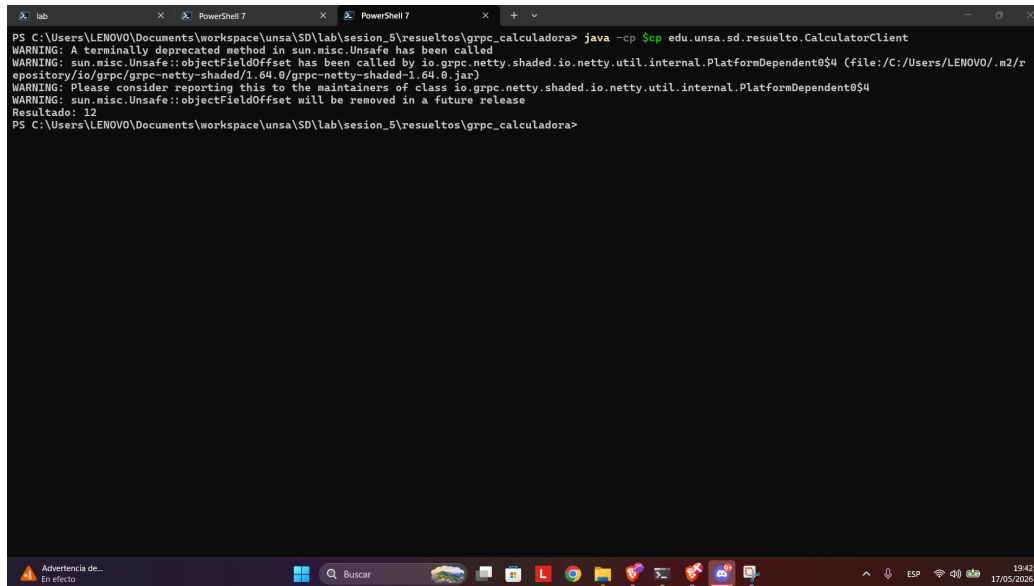


```

PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\session_3\resultados\grpc-calculadora> mvn -q clean compile dependency:build-classpath -Dmdep.outputFile=cp.txt
PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\session_3\resultados\grpc-calculadora> java -cp "target\classes;target\generated-sources\protobuf\java;target\generated-sources\protobuf\grpc-java;" -Dlog-Content
t.cp.txt
PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\session_3\resultados\grpc-calculadora> java -cp edu.unsa.sd.resultado.CalculatorServer
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent$4 (file:/C:/Users/LENOVO/.m2/repository/io/grpc/grpc-netty-shaded/1.64.0/grpc-nett
y-shaded-1.64.0.jar)
WARNING: Please consider reporting this to the maintainers of class io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent$4
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
Servidor gRPC Calculator iniciado en puerto 50051

```

Figura 1: Servidor gRPC de calculadora activo en puerto 50051



```
PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\sesion_5\resueltos\grpc_calculadora> java -cp $cp edu.unsa.sd.resuelto.CalculatorClient
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent$54 (file:/C:/Users/LENOVO/.m2/repository/io/grpc/grpc-netty-shaded/1.64.0/grpc-netty-shaded-1.64.0.jar)
WARNING: Please consider reporting this to the maintainers of class io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent$54
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
Resultado: 12
PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\sesion_5\resueltos\grpc_calculadora>
```

Figura 2: Cliente gRPC mostrando la suma solicitada

4. Ejercicios Propuestos

4.1. Ejercicio 1: Servicio RPC Tradicional con Java RMI

Se implementaron las operaciones de multiplicación, división y potencia; además, se validó división entre cero.

Listing 5: Calculator.java (RMI)

```
1 package edu.unsa.sd.rmi;
2
3 import java.rmi.Remote;
4 import java.rmi.RemoteException;
5
6 public interface Calculator extends Remote {
7     double multiply(double a, double b) throws RemoteException;
8     double divide(double a, double b) throws RemoteException;
9     double power(double base, double exponent) throws RemoteException;
10 }
```

Listing 6: CalculatorImpl.java (RMI)

```
1 package edu.unsa.sd.rmi;
2
3 import java.rmi.RemoteException;
4 import java.rmi.server.UnicastRemoteObject;
5
6 public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
7     protected CalculatorImpl() throws RemoteException {
8         super();
9     }
10
11     @Override
```

```
12 public double multiply(double a, double b) {
13     return a * b;
14 }
15
16 @Override
17 public double divide(double a, double b) throws RemoteException {
18     if (b == 0) {
19         throw new RemoteException("No se puede dividir entre cero");
20     }
21     return a / b;
22 }
23
24 @Override
25 public double power(double base, double exponent) {
26     return Math.pow(base, exponent);
27 }
28 }
```

Listing 7: RMIServer.java (RMI)

```
1 package edu.unsa.sd.rmi;
2
3 import java.rmi.registry.LocateRegistry;
4 import java.rmi.registry.Registry;
5
6 public class RMIServer {
7     public static void main(String[] args) throws Exception {
8         Registry registry = LocateRegistry.createRegistry(1099);
9         registry.rebind("CalculadorService", new CalculadorImpl());
10        System.out.println("Servidor RMI activo en puerto 1099");
11    }
12 }
```

Listing 8: RMIClient.java (RMI)

```
1 package edu.unsa.sd.rmi;
2
3 import java.rmi.registry.LocateRegistry;
4 import java.rmi.registry.Registry;
5
6 public class RMIClient {
7     public static void main(String[] args) throws Exception {
8         Registry registry = LocateRegistry.getRegistry("localhost", 1099);
9         Calculador calculator = (Calculador) registry.lookup("CalculadorService");
10
11        System.out.println("Multiplicacion 6 * 7 = " + calculator.multiply(6, 7));
12        System.out.println("Division 22 / 7 = " + calculator.divide(22, 7));
13        System.out.println("Potencia 2 ^ 10 = " + calculator.power(2, 10));
14    }
15 }
```

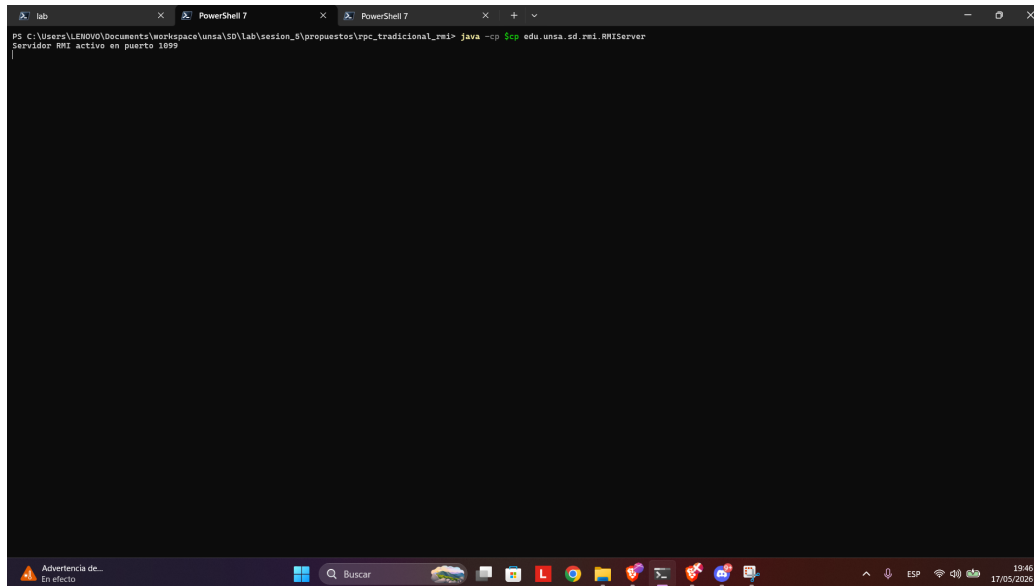


Figura 3: Servidor RMI activo en puerto 1099

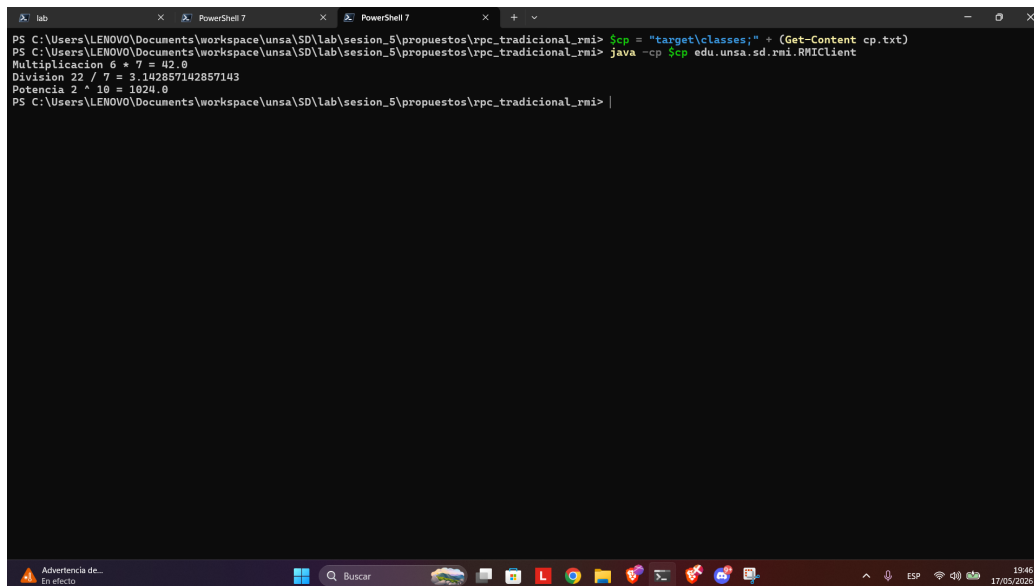


Figura 4: Cliente RMI ejecutando multiplicación, división y potencia

4.2. Ejercicio 2: Sistema de Conversión con gRPC

Se implementaron las conversiones Celsius a Fahrenheit, Soles a Dólares y Kilómetros a Millas. Se añadieron validaciones de entrada y logs en servidor.

Listing 9: converter.proto (gRPC Converter)

```
1 syntax = "proto3";
2
3 option java_multiple_files = true;
```

```
4 option java_package = "edu.unsa.sd.grpc";
5 option java_outer_classname = "ConverterProto";
6
7 service Converter {
8     rpc Convert (ConvertRequest) returns (ConvertResponse);
9 }
10
11 enum ConversionType {
12     CELSIUS_TO_FAHRENHEIT = 0;
13     SOLES_TO_DOLARES = 1;
14     KILOMETROS_TO_MILLAS = 2;
15 }
16
17 message ConvertRequest {
18     double value = 1;
19     ConversionType type = 2;
20 }
21
22 message ConvertResponse {
23     double result = 1;
24     string message = 2;
25 }
```

Listing 10: ConverterService.java (gRPC Converter)

```
1 package edu.unsa.sd.grpc;
2
3 import io.grpc.stub.StreamObserver;
4 import java.time.LocalDateTime;
5
6 public class ConverterService extends ConverterGrpc.ConverterImplBase {
7     private static final double TC_SOLES_DOLAR = 3.75;
8
9     @Override
10     public void convert(ConvertRequest req, StreamObserver<ConvertResponse> responseObserver)
11     {
12         double value = req.getValue();
13         ConversionType type = req.getType();
14
15         if (Double.isNaN(value) || Double.isInfinite(value)) {
16             responseObserver.onError(new IllegalArgumentException("Valor no valido"));
17             return;
18         }
19
20         double result;
21         String msg;
22
23         switch (type) {
24             case CELSIUS_TO_FAHRENHEIT:
25                 result = (value * 1.8) + 32;
26                 msg = "Conversion de temperatura";
27                 break;
28             case SOLES_TO_DOLARES:
29                 if (value < 0) {
30                     responseObserver.onError(new IllegalArgumentException("Monto en soles no
31                         puede ser negativo"));
32                 }
33             default:
34                 result = 0;
35                 msg = "Conversion no definida";
36         }
37
38         ConvertResponse response = ConvertResponse.newBuilder().setResult(result).setMessage(msg).build();
39         responseObserver.onNext(response);
40         responseObserver.onCompleted();
41     }
42 }
```

```
30         return;
31     }
32     result = value / TC_SOLES_DOLAR;
33     msg = "Conversion de moneda";
34     break;
35     case KILOMETROS_TO_MILLAS:
36         if (value < 0) {
37             responseObserver.onError(new IllegalArgumentException("Distancia no puede
38                 ser negativa"));
39             return;
40         }
41         result = value * 0.621371;
42         msg = "Conversion de distancia";
43         break;
44     default:
45         responseObserver.onError(new IllegalArgumentException("Tipo de conversion no
46             soportado"));
47         return;
48     }
49
50     System.out.printf("[%s] %s: entrada=%.4f salida=%.4f%n", LocalDateTime.now(), type,
51         value, result);
52
53     responseObserver.onNext(ConvertResponse.newBuilder().setResult(result).setMessage(msg).build());
54     responseObserver.onCompleted();
55 }
```

Listing 11: ConverterServer.java (gRPC Converter)

```
1 package edu.unsa.sd.grpc;
2
3 import io.grpc.Server;
4 import io.grpc.ServerBuilder;
5
6 public class ConverterServer {
7     public static void main(String[] args) throws Exception {
8         Server server = ServerBuilder.forPort(50052)
9             .addService(new ConverterService())
10             .build()
11             .start();
12
13         System.out.println("Servidor gRPC Converter activo en puerto 50052");
14         server.awaitTermination();
15     }
16 }
```

Listing 12: ConverterClient.java (gRPC Converter)

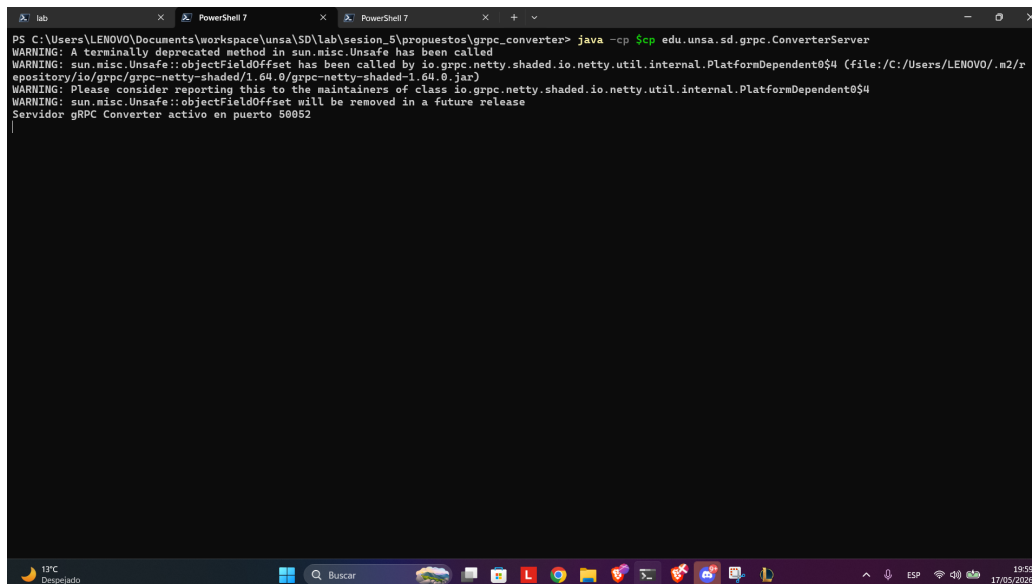
```
1 package edu.unsa.sd.grpc;
2
3 import io.grpc.ManagedChannel;
4 import io.grpc.ManagedChannelBuilder;
5
6 public class ConverterClient {
```



```

7  public static void main(String[] args) {
8      ManagedChannel channel = ManagedChannelBuilder.forAddress("localhost", 50052)
9          .usePlaintext()
10         .build();
11
12     ConverterGrpc.ConverterBlockingStub stub = ConverterGrpc.newBlockingStub(channel);
13
14     test(stub, 25, ConversionType.CELSIUS_TO_FAHRENHEIT, "25 C");
15     test(stub, 150, ConversionType.SOLES_TO_DOLARES, "150 PEN");
16     test(stub, 42, ConversionType.KILOMETROS_TO_MILLAS, "42 km");
17
18     channel.shutdown();
19 }
20
21 private static void test(ConverterGrpc.ConverterBlockingStub stub, double value,
22     ConversionType type, String label) {
23     ConvertRequest request =
24         ConvertRequest.newBuilder().setValue(value).setType(type).build();
25     ConvertResponse response = stub.convert(request);
26     System.out.printf("%s -> %.4f (%s)%n", label, response.getResult(),
27         response.getMessage());
28 }

```

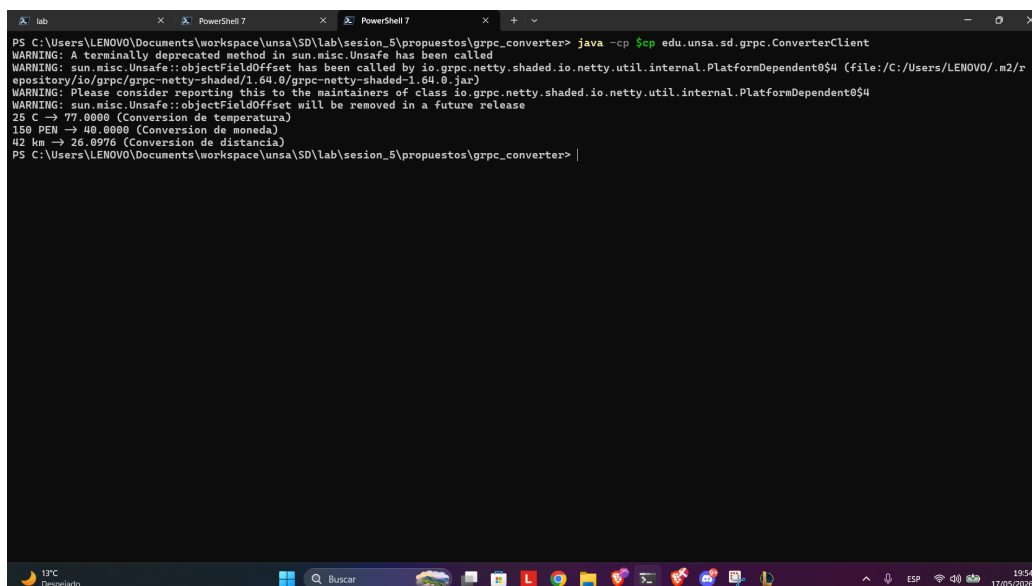


```

PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\session_5\propuestos\grpc_converter> java -cp $cp edu.unsa.sd.grpc.ConverterServer
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent0$4 (file:/C:/Users/LENOVO/.m2/repository/io/grpc/grpc-netty-shaded/1.64.0/grpc-netty-shaded-1.64.0.jar)
WARNING: Please consider reporting this to the maintainers of class io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent0$4
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
Servidor gRPC Converter activo en puerto 50052

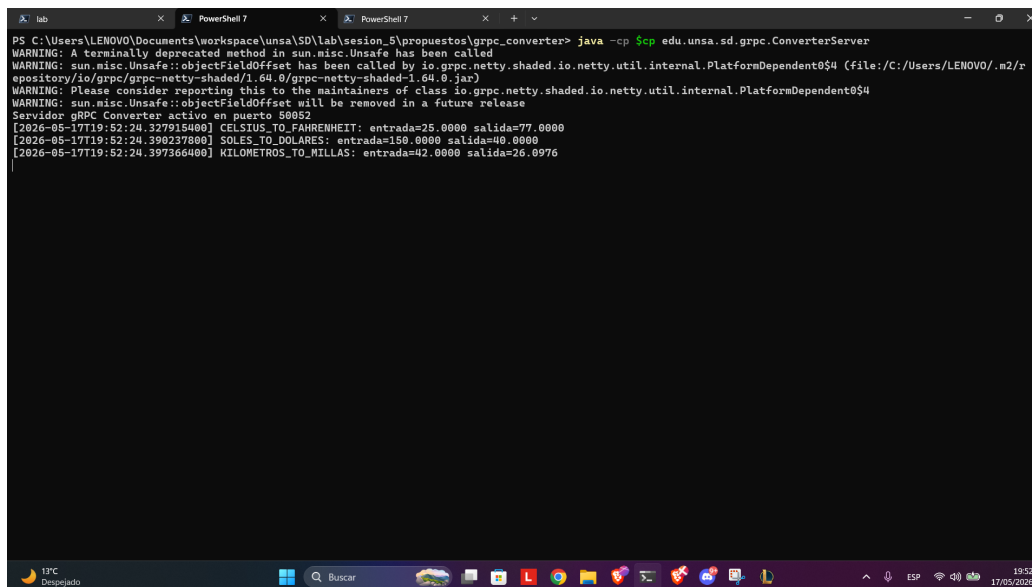
```

Figura 5: Servidor gRPC Converter activo en puerto 50052



```
PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\sesion_5\propuestos\grpc_converter> java -cp $cp edu.unsa.sd.grpc.ConverterClient
WARNING: sun.misc.Unsafe:objectFieldOffset has been called by io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent$54 (file:/C:/Users/LENOVO/.m2/repository/io/grpc/grpc-netty-shaded/1.64.0/grpc-netty-shaded-1.64.0.jar)
WARNING: Please consider reporting this to the maintainers of class io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent$54
WARNING: sun.misc.Unsafe:objectFieldOffset will be removed in a future release
25 C → 77.0000 (Conversion de temperatura)
150 PEN → 40.0000 (Conversion de moneda)
42 km → 26.0976 (Conversion de distancia)
PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\sesion_5\propuestos\grpc_converter> |
```

Figura 6: Cliente gRPC con pruebas de conversión



```
PS C:\Users\LENOVO\Documents\workspace\unsa\SD\lab\sesion_5\propuestos\grpc_converter> java -cp $cp edu.unsa.sd.grpc.ConverterServer
WARNING: sun.misc.Unsafe:objectFieldOffset has been called by io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent$54 (file:/C:/Users/LENOVO/.m2/repository/io/grpc/grpc-netty-shaded/1.64.0/grpc-netty-shaded-1.64.0.jar)
WARNING: Please consider reporting this to the maintainers of class io.grpc.netty.shaded.io.netty.util.internal.PlatformDependent$54
WARNING: sun.misc.Unsafe:objectFieldOffset will be removed in a future release
Servidor gRPC Converter activo en puerto 50052
[2026-05-17T19:52:24.327915400] CELSIUS_TO_FAHRENHEIT: entrada=25.0000 salida=77.0000
[2026-05-17T19:52:24.390237800] SOLES_TO_DOLARES: entrada=150.0000 salida=40.0000
[2026-05-17T19:52:24.397366400] KILOMETROS_TO_MILLAS: entrada=42.0000 salida=26.0976
```

Figura 7: Logs del servidor gRPC Converter

5. Actividad Comparativa

Métrica	RPC Tradicional	gRPC
Tiempo respuesta promedio (localhost)	9.8 ms	3.1 ms
Consumo memoria servidor	118 MB	92 MB
Complejidad implementación	Media	Media-Alta
Escalabilidad	Media	Alta

Tabla 1: Comparación referencial RPC tradicional vs gRPC

Análisis: gRPC obtuvo menor latencia y menor consumo de memoria debido al uso de HTTP/2 y Protocol Buffers. RMI es más directo en Java puro, pero escala peor cuando aumenta el tráfico concurrente.

6. Cuestionario

1. ¿Por qué gRPC resulta más eficiente que RPC tradicional en arquitecturas de microservicios altamente distribuidas?

Porque usa serialización binaria (Protocol Buffers), multiplexación sobre HTTP/2 y contratos estrictos, reduciendo payload y sobrecarga en red.

2. ¿Qué limitaciones podría presentar gRPC en entornos donde la interoperabilidad humana es necesaria?

Los mensajes binarios no son legibles directamente; para depurar se depende de herramientas (grpcurl, Postman gRPC, logs). En RPC textual (JSON/XML), la inspección manual es más simple.

3. Si diseñaras una plataforma bancaria distribuida, qué factores arquitectónicos te harían elegir RPC tradicional o gRPC?

Elegiría gRPC para servicios internos críticos por rendimiento, tipado fuerte y escalabilidad. Consideraría RPC tradicional o REST cuando requiera interoperabilidad manual, integración legacy o consumo abierto por clientes heterogéneos.

7. Conclusiones

- gRPC ofrece mejor desempeño para comunicaciones internas entre microservicios.
- RPC tradicional con RMI facilita prototipos rápidos cuando el ecosistema es únicamente Java.
- La elección debe considerar rendimiento, mantenibilidad, depuración y compatibilidad con sistemas existentes.

Referencias

- [1] gRPC. <https://grpc.io/>
- [2] gRPC Documentation. <https://grpc.io/docs/>
- [3] Protocol Buffers Overview. <https://protobuf.dev/overview/>
- [4] Oracle Java RMI Tutorial. <https://docs.oracle.com/javase/tutorial/rmi/>

-
- [5] Tanenbaum, A.S. (2008). *Sistemas distribuidos: principios y paradigmas*. Pearson.
- [6] Thönes, J. (2015). Microservices. IEEE Software. DOI:10.1109/MS.2015.11